Advanced Web Application Development Technologies

Lab

Dimitris Ringas

riggas@ionio.gr

Department of Informatics, Ionian University



HTTP and Python

Web Technologies & API Calls

(part 2)



Session Objectives

- **Pagination** of API results
- **Uploading and downloading** files over HTTP (multipart, streaming)
- **Rate limits** and request retries (retry, exponential backoff)
- **Logging** requests and responses for debugging
- **CRUD** operations on RESTful APIs
- **Authentication**: API Keys, Bearer Tokens, Basic Auth

Paginated APIs

- **APIs often do not return all the data in a single call**, especially when there is a lot of it (e.g. thousands of records).
- For **performance** and **network load** reasons, results are split into *pages*.
- The client (e.g. a Python application) must make **multiple requests** to get all the data.
- Typical parameters:
  - `page` or `offset`: which page we ask for
  - `per_page` or `limit`: how many items per page
  - or a cursor, e.g. `since`: "give me whatever comes after id X"
- The server may return a **`Link` header** with links to the next/previous pages (`rel="next"`, `rel="last"`, ...). `requests` parses it automatically into `response.links`.
  - open https://api.github.com/users/octocat/repos?per_page=3 and look at the `Link` response header in your browser's Dev Tools
  - run the code once without the `while` loop and print the `Link` header
  - run the code and see how many pages the GitHub API returns.

```python
import requests

def fetch_all_pages(url, params=None, max_pages=5):
    items = []
    pages = 0

    while url and pages < max_pages:
        response = requests.get(url, params=params, timeout=10)
        response.raise_for_status()

        items.extend(response.json())
        pages += 1
        print(f"Page {pages}: {response.url}")

        # requests parses the Link header automatically
        url = response.links.get('next', {}).get('url')
        params = None  # The next URL already contains the parameters

    return items

# Usage
repos = fetch_all_pages('https://api.github.com/users/octocat/repos',
                        params={'per_page': 3})
print(f"Total repos: {len(repos)}")
```

File Upload and Download

Exchanging files over HTTP is a basic communication mechanism between client and server.

- **File Upload:**
  The client sends a file to the server, usually with an HTTP `POST` request of type `multipart/form-data`.
  - The data is sent as multipart/form-data (not plain JSON).
  - Each entry of the files dictionary becomes a separate part of the multipart request.
  - The server must have an endpoint able to handle uploads (e.g. Flask: request.files).
- **File Download:**
  The client requests a file from the server with an HTTP `GET` request.
  The file can be sent **in one piece** or **in chunks (streaming)** for large sizes.
  - response.content holds the entire response body (suitable for small files).
  - With stream=True the file is downloaded in chunks → more efficient for large files.
  - Useful for large downloads, such as backups or media.

File Upload

```python
import requests

# Single file upload
with open('document.pdf', 'rb') as file:
    files = {'file': file}
    response = requests.post('https://httpbin.org/post', files=files, timeout=10)

# Multiple files
with open('file1.txt', 'rb') as f1, open('file2.txt', 'rb') as f2:
    files = {'file1': f1, 'file2': f2}
    response = requests.post('https://httpbin.org/post', files=files, timeout=10)
```

File Download

```python
import requests

# Small files
response = requests.get('https://httpbin.org/image/png', timeout=10)
response.raise_for_status()
with open('downloaded_image.png', 'wb') as file:
    file.write(response.content)

# Large files (streaming) - example.com/largefile.zip is a placeholder URL
with requests.get('https://example.com/largefile.zip', stream=True, timeout=10) as response:
    response.raise_for_status()
    with open('largefile.zip', 'wb') as file:
        for chunk in response.iter_content(chunk_size=8192):
            file.write(chunk)
```

Rate Limiting and Retry Logic

Most APIs enforce **limits on the request rate** (*Rate Limits*) to protect themselves from excessive use or abuse.

- **Rate Limit:** The maximum number of requests allowed within a given time window
  e.g. *60 requests per minute*.
- If the client exceeds this limit, the server returns:
  `HTTP 429 Too Many Requests`
- This often comes with headers such as:
  - `X-RateLimit-Limit: 60`: the total number of requests allowed within a time window
  - `X-RateLimit-Remaining: 0`: how many requests are left in the current window
  - `X-RateLimit-Reset: 1731201120`: when the limit resets, as a Unix timestamp
  - `Retry-After: 30`: how many seconds the client must wait before trying again
- The client *must* **wait** before repeating requests.
- On temporary errors (5xx, timeouts) we retry with **exponential backoff** (1s, 2s, 4s, ...). For errors such as 400/401/404 retrying makes *no* sense.

E.g.: GitHub API → 60 requests/hour per IP without authentication, 5000/hour with a token.

```python
import requests
import time

RETRYABLE = {429, 500, 502, 503, 504}

def api_call_with_retry(url, max_retries=3):
    for attempt in range(max_retries + 1):
        try:
            response = requests.get(url, timeout=10)
        except (requests.exceptions.ConnectionError,
                requests.exceptions.Timeout) as e:
            if attempt == max_retries:
                raise
            print(f"Attempt {attempt + 1} failed ({e}). Retrying...")
            time.sleep(2 ** attempt)  # Exponential backoff
            continue

        if response.status_code in RETRYABLE and attempt < max_retries:
            # Retry-After (if present), otherwise exponential backoff
            wait = int(response.headers.get('Retry-After', 2 ** attempt))
            print(f"Status {response.status_code}. Waiting {wait}s...")
            time.sleep(wait)
            continue

        response.raise_for_status()  # 4xx/5xx that are not retried
        return response
```

Logging and Debugging

Logging requests is essential for debugging, analysing and monitoring the behaviour of an API client.

- It lets us see **which requests the client sends** and **what the server replies**.
- It is particularly useful for:
  - Analysing errors (timeouts, 4xx, 5xx)
  - Spotting unexpected data
  - Monitoring performance (response times)
- `logging` is Python's **built-in library** for log messages.
- The `requests` library can **produce detailed logs** when configured accordingly.
- Every `response` object contains both the **request** and the **response**, so we can display them together.

The logging system has several levels:

Level	Function	Use
DEBUG	`logger.debug()`	Detailed information (dev only)
INFO	`logger.info()`	Normal operation (e.g. successful request)
WARNING	`logger.warning()`	Minor problem or unexpected situation
ERROR	`logger.error()`	Error during execution
CRITICAL	`logger.critical()`	Serious error – needs immediate attention

```python
import requests
import logging

# Logging setup with timestamps
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(message)s',
    handlers=[logging.FileHandler('requests.log'), logging.StreamHandler()]
)

logger = logging.getLogger(__name__)

def log_request_response(response):
    logger.info(f"REQUEST:  {response.request.method} {response.request.url}")
    logger.debug(f"Request headers: {response.request.headers}")

    logger.info(f"RESPONSE: {response.status_code} "
                f"in {response.elapsed.total_seconds():.2f}s")
    logger.debug(f"Response headers: {response.headers}")
    logger.debug(f"Body (first 500 chars): {response.text[:500]}")

response = requests.get('https://httpbin.org/json', timeout=10)
log_request_response(response)
```

- With `level=logging.INFO` the `logger.debug()` calls are **not shown**; switch to `logging.DEBUG` to see headers & body (and the `urllib3` logs).
- print() always writes to stdout, with no severity levels.
  - To silence them you have to delete or comment them out.
- logging is more flexible:
  - Filtering by level without code changes, timestamps, module name.
  - It can write to files, syslog, a database, a remote server etc.

CRUD Operations on APIs

CRUD operations (Create, Read, Update, Delete) are the **foundation of every RESTful API**. Each HTTP method corresponds to an action on *resources*.

Action	HTTP Method	Description	Success code
Create	`POST`	Creates a new resource	`201 Created`
Read	`GET`	Reads data	`200 OK`
Update	`PUT`	Fully updates an existing resource	`200 OK`
Partial Update	`PATCH`	Updates part of a resource	`200 OK`
Delete	`DELETE`	Deletes a resource	`204 No Content` (or `200 OK`)

Example with a real RESTful API

- Use JSONPlaceholder, a free *mock API* for testing.
- It offers endpoints such as `/posts`, `/users`, `/comments`.
- It accepts `POST/PUT/PATCH/DELETE` without actually changing any data.
  - Every new post gets `id: 101`.
  - A `GET` after the `PUT` returns the original data.
- Note the status codes: `POST` → `201`, `DELETE` → `200` (not `204`).

```python
import requests

BASE_URL = 'https://jsonplaceholder.typicode.com'

# CREATE (POST)
new_post = {
    'title': 'My New Post',
    'body': 'This is the content of my post',
    'userId': 1
}
response = requests.post(f'{BASE_URL}/posts', json=new_post, timeout=10)
created_post = response.json()
print(f"{response.status_code} Created post with ID: {created_post['id']}")

# READ (GET)
response = requests.get(f'{BASE_URL}/posts/1', timeout=10)
post = response.json()

# UPDATE (PUT)
post['title'] = 'Updated Title'
response = requests.put(f'{BASE_URL}/posts/1', json=post, timeout=10)

# PARTIAL UPDATE (PATCH)
update_data = {'title': 'Patched Title'}
response = requests.patch(f'{BASE_URL}/posts/1', json=update_data, timeout=10)

# DELETE
response = requests.delete(f'{BASE_URL}/posts/1', timeout=10)
print(f"Delete status: {response.status_code}")
```

Authentication Methods

Most APIs require **authentication** to protect their data and to make sure requests come from **trusted clients**.

Authentication serves three main goals:

1. **Identifying** the client or user
2. **Access control** to the allowed endpoints
3. **Usage tracking** (logging, rate limiting, analytics)

Authentication Methods → API Keys

API Keys are simple "strings" assigned to each user or application.
They are usually included:

- either in the HTTP headers
- or as a query parameter (?api_key=...)

They provide basic identification but not strong security (they leak easily if they end up in logs or URLs).
Ideal for public APIs or simple machine-to-machine communication.

⚠️ Never write the key in your code or push it to GitHub; read it from an **environment variable**.

```python
import os
import requests

API_KEY = os.getenv('MY_API_KEY')  # export MY_API_KEY=...

# Header-based API Key
headers = {'X-API-Key': API_KEY}
response = requests.get('https://api.service.com/data', headers=headers)

# Query parameter API Key (e.g. OpenWeatherMap: ?appid=...)
params = {'api_key': API_KEY}
response = requests.get('https://api.service.com/data', params=params)
```

Authentication Methods → Bearer Token Authentication

Used in more secure, modern APIs.
Bearer tokens (e.g. JWT or OAuth 2.0 access tokens):

- are issued after a login/authorization process,
- are sent in the header `Authorization: Bearer <token>`.

Each token has an expiry date and can be revoked.
Provides strong security and granular permissions per user or application.

Try it without an account: https://httpbin.org/bearer returns `200` with any token and `401` without one.

```python
import os
import requests

# e.g. a GitHub Personal Access Token
# (Settings → Developer settings → Personal access tokens)
token = os.getenv('GITHUB_TOKEN')

headers = {'Authorization': f'Bearer {token}'}
response = requests.get('https://api.github.com/user',
                        headers=headers, timeout=10)
print(response.status_code, response.json().get('login'))
print("Limit:", response.headers.get('X-RateLimit-Limit'))  # 5000

# Test with httpbin
response = requests.get('https://httpbin.org/bearer',
                        headers={'Authorization': 'Bearer test-token'},
                        timeout=10)
print(response.json())  # {'authenticated': True, 'token': 'test-token'}
```

Authentication Methods → Basic Authentication

Basic Auth sends the credentials (username:password) Base64-encoded in an HTTP header.
Simple, but not secure without HTTPS, because it can be intercepted.
Still widely used in:

- internal systems or testing APIs
- CI/CD tools and scripts

Base64 ≠ encryption: `dXNlcjpwYXNzd2Q=` decodes instantly to `user:passwd`.

```python
import requests
from requests.auth import HTTPBasicAuth

# httpbin accepts the user/passwd given in the URL
url = 'https://httpbin.org/basic-auth/user/passwd'

response = requests.get(url, auth=HTTPBasicAuth('user', 'passwd'), timeout=10)
print(response.status_code, response.json())

# Shorthand syntax
response = requests.get(url, auth=('user', 'passwd'), timeout=10)

# The header that was sent
print(response.request.headers['Authorization'])  # Basic dXNlcjpwYXNzd2Q=

# Wrong password → 401 Unauthorized
print(requests.get(url, auth=('user', 'wrong'), timeout=10).status_code)
```

😎 That's enough.. *for today*!

Continue at:

https://github.com/riggas-ionio/web-II/

